Dependency Injection

DI · Wire Greeter without hard `new` in controllers · Build with Waqar

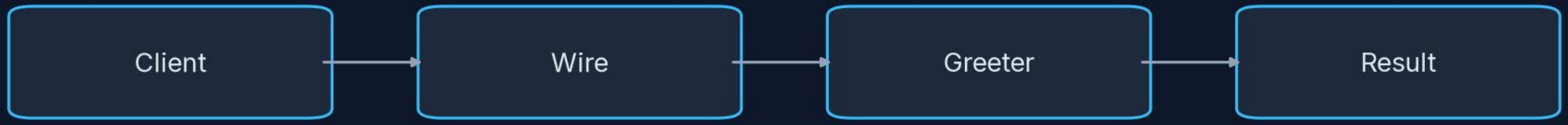
Today's Hook

- Problem: Wire Greeter without hard `new` in controllers
- Pattern: DI — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Wire Greeter without hard `new` in controllers
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



DI: Wire Greeter without hard `new` in controllers

Real-World Scenario

- Running example: Wire Greeter without hard `new` in controllers
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.di;

import com.buildwithwaqar.patterns.enterprise.di.*;
/** Problem: wire Greeter without hard-coding FriendlyGreeter in controllers. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== DI problem: wire greeter ===");
    var c = new DiContainer();
    c.singleton(Greeter.class, FriendlyGreeter::new);
    System.out.println(c.get(Greeter.class).greet("Waqar"));
    System.out.println("same=" + (c.get(Greeter.class) == c.get(Greeter.class)));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.di;

import java.util.HashMap;
import java.util.Map;
import java.util.Objects;
import java.util.function.Supplier;

/** Tiny DI / service locator for demos — prefer a real container in production. */
public final class DiContainer {
    private final Map<Class<?>, Supplier<?>> factories = new HashMap<>();
    private final Map<Class<?>, Object> singletons = new HashMap<>();

    public <T> void bind(Class<T> type, Supplier<? extends T> factory) {
        factories.put(type, factory);
    }

    public <T> void singleton(Class<T> type, Supplier<? extends T> factory) {
        factories.put(type, () -> singletons.computeIfAbsent(type, k -> factory.get()));
    }

    @SuppressWarnings("unchecked")
    public <T> T get(Class<T> type) {
        var f = factories.get(type);
        if (f == null) throw new IllegalArgumentException("unbound " + type.getName());
        return (T) Objects.requireNonNull(f.get());
    }
```

Solution (2)

```
}

interface Greeter { String greet(String name); }
record FriendlyGreeter() implements Greeter {
    public String greet(String name) { return "hello " + name; }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.enterprise.di;

import com.buildwithwaqar.patterns.enterprise.di.*;
/** Problem: wire Greeter without hard-coding FriendlyGreeter in controllers. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== DI problem: wire greeter ===");
    var c = new DiContainer();
    c.singleton(Greeter.class, FriendlyGreeter::new);
    System.out.println(c.get(Greeter.class).greet("Waqar"));
    System.out.println("same=" + (c.get(Greeter.class) == c.get(Greeter.class)));
  }
}
```

Walkthrough Output

```
=== DI problem: wire greeter ===
hello Waqar
same=true
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with DI.

Related Patterns

- Contrast DI with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → DI → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.